

Summary:

① Deletion & Insertion of on linked lists on:

- ① Head
- ② Tail
- ③ k^{th} Node
- ④ By value.

```

Node* insertByValue(Node* head, int value) {
    Node* temp = head;
    if (temp == NULL) {
        if (head == NULL) {
            Node* newNode = new Node(value);
            head = temp = newNode;
            return newNode;
        }
    }
    while (temp != NULL) {
        if (temp->next == NULL) {
            Node* newNode = new Node(value);
            temp->next = newNode;
            return newNode;
        }
        temp = temp->next;
    }
}
    
```

```

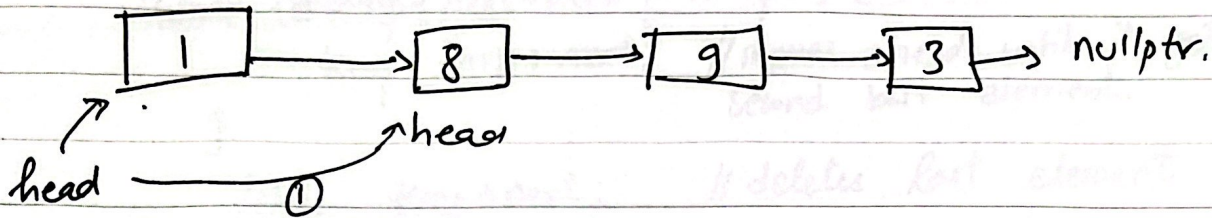
while (temp != NULL) {
    if (temp->next == NULL) {
        Node* newNode = new Node(value);
        temp->next = newNode;
        return newNode;
    }
    temp = temp->next;
}
    
```

```

Node* insertBetween(Node* head, int value, Node* after) {
    Node* temp = head;
    if (temp == NULL) {
        return head;
    }
    while (temp != NULL) {
        if (temp == after) {
            Node* newNode = new Node(value);
            temp->next = newNode;
            newNode->next = temp->next;
            return head;
        }
        temp = temp->next;
    }
    return head;
}
    
```

DELETING IN LL

→ Deleting the head of the linked list.

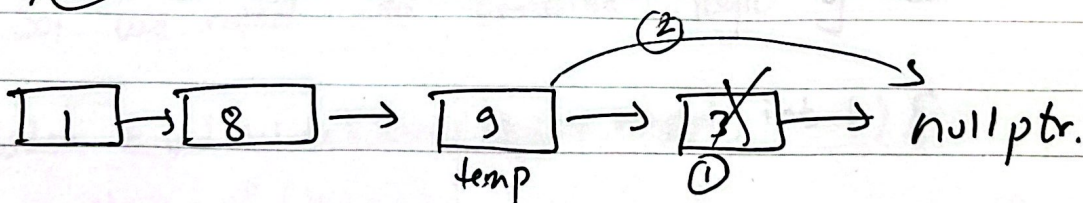


① Move head to next

② Delete previous head. We may use a temp variable for that.
So,

```
Node* removeFirstNode(Node* head) {  
    if (head == nullptr) return head;  
    Node* temp = head; // temp = head  
    head = head->next; // moves head 1 step ahead  
    delete head; temp; // deletes head [∵ temp = head]  
    return head; // returns new head.  
}
```

Deleting the tail of the linked list:



① traverse until you get to second last element i.e. 9. using temp. [while (temp->next->next != nullptr)]

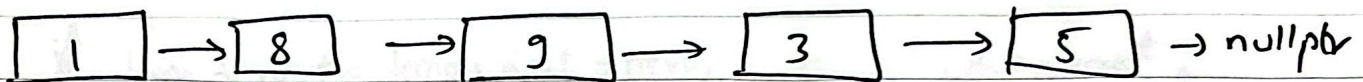
② Delete temp->next and set temp->next = nullptr.

```

Node* deleteTail(Node* head) {
    if (head == nullptr || head->next == nullptr) return null;
    Node* temp = head;
    while (temp->next->next != nullptr) {
        temp = temp->next; // moves ahead until it gets to
                           // second last element.
    }
    delete temp->next; // deletes last element
    temp->next = nullptr; // points to nullptr
    return head;
}

```

Deleting the k^{th} term in a linked list.



There might be 3 cases:

- ① k can be 1 [So delete head]
- ② k can be last [so delete tail]
- ③ k can be in between.

So, we need to combine logic of all.

```

Node* head deleteKItem(Node* head, int k) {

```

```

    Node* if (k == 1) {

```

```

        Node* temp = head;

```

```

        head = head->next;

```

```

        delete temp;

```

```

        return head;
    }

```

```

}

```

```

if

```

} logic for no. ①

```

Node* temp = head;
for (int i = 1; i < k; i++) {
    if (temp == nullptr || temp->next == nullptr) return head; //invalid k
    temp = temp->next;
}

```

```

Node* itemToDelete = temp->next;

```

```

if (itemToDelete->next == nullptr) {
    temp->next = nullptr;
    delete itemToDelete;
    return head;
} // if deleting tail

```

```

temp->next = temp->next->next;
delete itemToDelete;
return head;
} // if kth element (middle node)

```

Remove by Value

Same logic as removing k^{th} element. Just need to check if $\text{temp} \rightarrow \text{data} == \text{element}$.



keep temp one step ^{ahead} behind of value. by:

if (temp->next == data) {

then,
~~temp->next~~ itemToDelete = temp->next.

Delete itemToDelete.

In Code:

Node* deleteByValue (Node* head, int Value) {

if (head == nullptr) return nullptr;

if (head->data == value) {

Node* temp = head;

head = head->next;

delete temp;

return head;

while (~~temp->next->data == value~~ temp != nullptr & temp->next != nullptr)

if (temp->next->data == value) {

Node* itemToDelete = temp->next;

temp->next = temp->next->next;

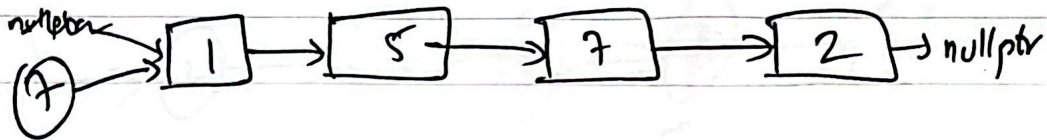
delete itemToDelete;

return head;

}

Insertion in linked list

① Inserting at head:

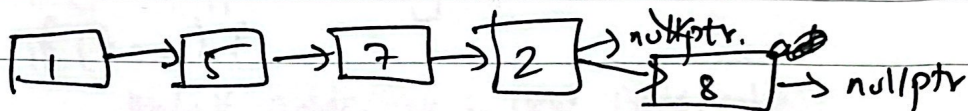


① Make a node with data=7 and next=1.

Code:

```
Node* insertHead(Node* head, int value) {  
    Node* temp = new Node(7, head);  
    return temp;  
}
```

② Inserting at the last (tail):



- ① traverse temp to last element
- ② Create a node to add. let's say: addedNode with ~~value~~ data = 8
- ③ link temp to addedNode next = nullptr.

```
Node* insertatTail(Node* head, int value) {
```

```
    Node* temp = head;
```

```
    while (temp->next != nullptr) {
```

```
        temp = temp->next;
```

```
    }
```

```
    Node* addedNode = new Node(value, nullptr);
```

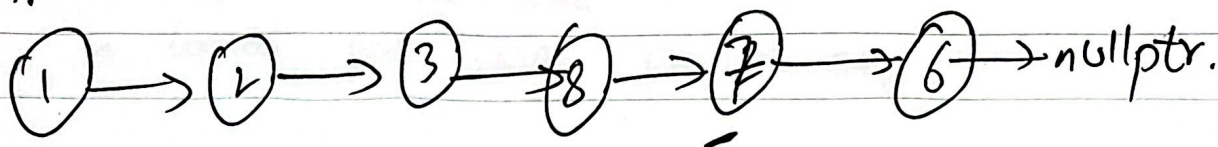
```
    temp->next = addedNode;
```

```
    return head;
```

g =

Insertion at kth term/Node

Suppose 5:



- loop till you get till 4th ~~th~~ Node.

then: add a Node

Connect Node $\rightarrow$ by: $\text{addedItem} \rightarrow \text{next} = \text{temp} \rightarrow \text{next};$
 $\text{temp} \rightarrow \text{next} = \text{addedItem};$

In code:

```
Insert at kth position(Node* head, int k, int value) {  
    if(k < 1 || k > sizeOfLinkedList + 1) return nullptr;  
    if(k == 1) {  
        Node* addedNode = new Node(value);  
        addedNode->next = head;  
        return addedNode;  
    }  
    Node* temp = head;  
    for(int i = 1; i <= k - 2; i++) temp = temp->next;  
    Node* addedNode = new Node(value);  
    addedNode->next = temp->next;  
    temp->next = addedNode;  
    return head;  
}
```

Inserting by Value:

- loop through until you find ~~Node → data~~ = Node → next → data = value.

→ Insert a new node

→ Connect node using previous step.

In code:

```
Node* InsertbyValue(Node* head, int value, int added item) {
```

```
    Node* temp = head;
```

```
    if (head == nullptr) return nullptr;
```

```
    if (head → data == value) {
```

```
        Node* addedNode = new Node(added item);
```

```
        addedNode → next = head;
```

```
        return addedNode;
```

```
    }
```

} if head
is value

```
    while (temp != nullptr && temp → next != nullptr) {
```

```
        if (temp → next → data == value) {
```

```
            Node* addedNode = new Node(added item);
```

```
            addedNode → next = temp → next;
```

```
            temp → next = addedNode;
```

```
            return head;
```

```
        }
```

```
        temp = temp → next;
```

```
    }
```

```
    return head;
```

} if value
is in
between
(head, tail]